

Trabajo Práctico Integrador

Investigación Operativa

Año 2024

Consigna

Optimización de la Gestión de Inventarios en una Panadería Francesa

Como parte de un proyecto de optimización de procesos en una panadería francesa con una única sucursal, se nos solicita que desarrollemos un modelo cuantitativo para la gestión de sus inventarios.

Objetivos:

- Identificar los insumos críticos para la producción de los productos más vendidos.
- Desarrollar modelos de pronóstico apropiados para la demanda de estos insumos.
- Diseñar una política de reabastecimiento óptima que minimice los costos totales de inventario.

Datos: Se proporcionan datos históricos de ventas de la panadería (<https://www.kaggle.com/datasets/matthieugimbert/french-bakery-daily-sales?authuser=1>).

Tareas:

- 1. Análisis Exploratorio:**
 - Limpieza y transformación de los datos.
 - Identificación de patrones estacionales y tendencias en las ventas.
 - Visualización de los datos para una mejor comprensión.
- 2. Modelado de Series de Tiempo:**
 - Selección de los insumos más relevantes.
 - Construcción de series de tiempo para cada insumo.
 - Ajuste de modelos de pronóstico utilizando Python (librería Statsmodels).
 - Evaluación de la precisión de los modelos.
- 3. Optimización de Inventarios:**
 - Desarrollo de modelos de inventario (alguna variante de modelo de cantidad económica de pedido) considerando los costos de adquisición, orden, y mantenimiento.
 - Implementación de los modelos en LINGO o Python (librería a elección).

- Determinación de los niveles óptimos de inventario y los puntos de reorden.

4. **Análisis de Sensibilidad:**

- Evaluación de cómo varían los resultados al modificar los parámetros de los modelos.

Entrega: Se deberá entregar un informe que incluya:

- Un análisis exhaustivo de los datos.
- La descripción de los modelos desarrollados y su justificación.
- Los resultados obtenidos y su interpretación.
- Un conjunto de recomendaciones para la mejora de la gestión de inventarios de la panadería.
- El código utilizado para realizar el análisis y la implementación de los modelos.

Consideraciones Adicionales:

- **Visualización:** Se recomienda utilizar librerías como Matplotlib o Seaborn para crear visualizaciones claras y concisas de los datos y los resultados.
- **Documentación:** Es importante documentar el código de manera clara y concisa, utilizando comentarios y docstrings.
- **Presentación:** El informe debe ser bien estructurado y fácil de comprender, con un lenguaje técnico adecuado.

Procesamiento de los datos

Conocimiento de los datos

En primer lugar, debemos conocer con qué tipo de datos estamos trabajando. Esta base de datos incluye información de las ventas diarias de una panadería, con detalles sobre los productos vendidos, cantidades y precios unitarios. Estos datos permitirán analizar patrones de demanda, clasificar productos según su importancia económica y optimizar la gestión de inventarios.

Para conocer la estructura de la base de datos, utilizaremos el siguiente código

```
import pandas as pd
file_path = './Bakery sales.csv'
bakery_sales = pd.read_csv(file_path)
bakery_sales.head(), bakery_sales.info()
```

De lo cuál obtenemos que la base de datos cuenta con 234005 registros y 7 columnas, las cuáles son:

- Unnamed: 0: Funciona como un ID.
- date: Fecha de la venta.
- time: Hora de la venta.
- ticket_number: Número del ticket de venta.
- article: Nombre del producto vendido.
- Quantity: Cantidad vendida de cada artículo.
- unit_price: Precio unitario del producto.

Limpieza de los datos

Para la limpieza de los datos vamos a tomar las siguientes acciones:

- **Estandarizar los nombres de los atributos** (formato minúsculas separadas por _)

```
bakery_sales.columns = bakery_sales.columns.str.lower().str.replace(' ', '_')
```

- **Eliminar atributo 'Unnamed: 0'**

```
bakery_sales.drop(columns=['unnamed: 0'], inplace=True)
```

- **Convertir el atributo 'date' a formato fecha**

```
bakery_sales['date'] = pd.to_datetime(bakery_sales['date'], format='%Y-%m-%d')
```

- **Convertir el atributo 'unit_price' a formato numérico**

```
bakery_sales['unit_price'] = bakery_sales['unit_price'].str.replace('€',  
'').str.replace(',', '.').astype(float)
```

- **Eliminar los valores nulos** (aunque no se encontraron valores nulos según la inspección inicial, podemos revisar en detalle)

```
bakery_sales.dropna(inplace=True)
```

- **Eliminar registros donde 'quantity' o 'unit_price' sean igual a 0.**

```
bakery_sales = bakery_sales[(bakery_sales['quantity'] > 0) &
(bakery_sales['unit_price'] > 0)]
```

- **Eliminar registros donde 'article' está vacío o es igual a un punto.**

```
bakery_sales = bakery_sales[~bakery_sales['article'].str.strip().isin(['',
'.'])]
```

- **Detectar outliers en el atributo 'Quantity'** (utilizando el rango intercuartil, permitimos solo aquellos dentro de 3 veces el rango intercuartil por debajo o por encima de Q1 y Q3).

```
q1_quantity = bakery_sales['quantity'].quantile(0.25)
q3_quantity = bakery_sales['quantity'].quantile(0.75)
iqr_quantity = q3_quantity - q1_quantity

bakery_sales = bakery_sales[
    (bakery_sales['quantity'] >= (q1_quantity - 3 * iqr_quantity)) &
    (bakery_sales['quantity'] <= (q3_quantity + 3 * iqr_quantity))
]
```

- **Crear una nueva columna 'revenue'** (calculando la cantidad por el precio unitario)

```
bakery_sales['revenue'] = bakery_sales['quantity'] *
bakery_sales['unit_price']
```

Análisis ABC

A partir del dataset con los datos limpios vamos a proceder con el análisis ABC, el cuál nos permitirá detectar cuales son los productos más vendidos y clasificarlos según el volumen de ventas acumulado y de esta forma identificar los productos que deben ser prioridad para la panadería.

En primer lugar, utilizando la columna 'revenue' que fue calculada anteriormente como $quantity \times unit_price$, vamos a ordenarlos en forma descendente por ingresos generados. Luego calcularemos dos nuevas columnas una que representa el ingreso acumulado y otra el porcentaje acumulado, de esta forma podremos ver qué porcentaje de los ingresos totales representa cada producto.

Y por último usaremos la regla de Pareto para determinar la clase de cada producto

- **Clase A:** Productos que generan aproximadamente el 80% de los ingresos.
- **Clase B:** Productos que representan el siguiente 15%.
- **Clase C:** El resto (~5%).

Quedando el código de la siguiente manera:

```
import pandas as pd
file_path = './Bakery sales- Clean.csv'
bakery_sales = pd.read_csv(file_path)

# Calcular ingresos totales por producto
product_revenue =
bakery_sales.groupby('article')['revenue'].sum().reset_index()

# Ordenar productos por ingresos totales en orden descendente
product_revenue.sort_values(by='revenue', ascending=False, inplace=True)

# Calcular el ingreso acumulado y el porcentaje respecto al total
total_revenue = product_revenue['revenue'].sum()
product_revenue['cumulative_revenue'] = product_revenue['revenue'].cumsum()
product_revenue['cumulative_percentage'] =
(product_revenue['cumulative_revenue'] / total_revenue) * 100

# Clasificar productos en clases A, B y C
def classify_abc(cumulative_percentage):
    if cumulative_percentage <= 80:
        return 'A'
    elif cumulative_percentage <= 95:
        return 'B'
    else:
        return 'C'

product_revenue['ABC_class'] =
product_revenue['cumulative_percentage'].apply(classify_abc)

product_revenue.to_csv('Analisis ABC.csv', index=False)
```

A partir del resultado de la ejecución obtenemos los siguientes resultados:

- **Clase A:** 26 productos que representan el 80% de los ingresos totales.
- **Clase B:** 31 productos que representan el siguiente 15% de los ingresos.
- **Clase C:** 90 productos que generan el 5% restante

Los productos más importantes (clase A) son los siguientes:

- **TRADITIONAL BAGUETTE** generó un ingreso de **\$137,196.55**.
- **FORMULE SANDWICH** generó un ingreso de **\$35,090.00**.
- **CROISSANT** generó un ingreso de **\$26,514.65**.
- **PAIN AU CHOCOLAT** generó un ingreso de **\$26,025.05**.
- **BANETTE** generó un ingreso de **\$23,806.65**.
- **BAGUETTE** generó un ingreso de **\$19,438.25**.
- **SPECIAL BREAD** generó un ingreso de **\$13,440.60**.
- **SANDWICH COMPLET** generó un ingreso de **\$13,345.00**.
- **TRAITEUR** generó un ingreso de **\$12,368.40**.
- **GRAND FAR BRETON** generó un ingreso de **\$10,730.00**.
- **TARTELETTE** generó un ingreso de **\$9,660.60**.
- **BRIOCHE** generó un ingreso de **\$9,274.00**.
- **VIK BREAD** generó un ingreso de **\$9,254.70**.
- **CEREAL BAGUETTE** generó un ingreso de **\$8,946.65**.
- **GD KOUIGN AMANN** generó un ingreso de **\$8,067.50**.
- **CAMPAGNE** generó un ingreso de **\$7,963.40**.
- **BOULE 400G** generó un ingreso de **\$7,346.50**.
- **MOISSON** generó un ingreso de **\$6,948.30**.
- **ECLAIR** generó un ingreso de **\$6,815.70**.
- **SAND JB EMMENTAL** generó un ingreso de **\$6,646.00**.
- **COMPLET** generó un ingreso de **\$5,779.70**.
- **KOUIGN AMANN** generó un ingreso de **\$4,820.50**.
- **PAIN BANETTE** generó un ingreso de **\$4,418.90**.
- **DIVERS VIENNOISERIE** generó un ingreso de **\$4,293.75**.
- **FINANCIER X5** generó un ingreso de **\$4,130.00**.
- **PAIN AUX RAISINS** generó un ingreso de **\$3,961.30**.

Series de tiempo

Por último, generamos un gráfico de series de tiempo, lo que nos permitirá visualizar la demanda y detectar posibles patrones o tendencias.

Quedándonos el código de la siguiente manera:

```
import matplotlib.pyplot as plt
import pandas as pd

product_revenue = pd.read_csv('./Analisis ABC.csv')
bakery_sales = pd.read_csv('./Bakery sales- Clean.csv')

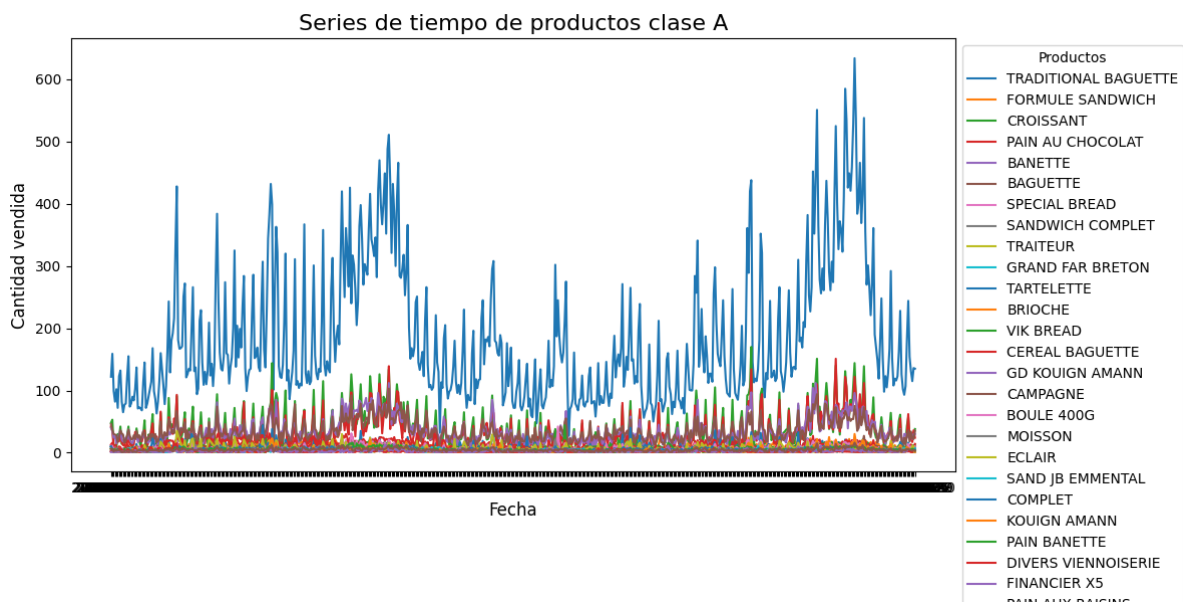
# Filtrar productos de clase A
class_a_products = product_revenue[product_revenue['ABC_class'] ==
'A']['article']
class_a_sales = bakery_sales[bakery_sales['article'].isin(class_a_products)]
```

```
# Agrupar las ventas diarias por producto
daily_sales = class_a_sales.groupby(['date',
'product'])['quantity'].sum().reset_index()

# Graficar las series de tiempo de los productos de clase A
plt.figure(figsize=(12, 6))

for product in class_a_products:
    product_data = daily_sales[daily_sales['product'] == product]
    plt.plot(product_data['date'], product_data['quantity'], label=product)

# Personalizar el gráfico
plt.title('Series de tiempo de productos clase A', fontsize=16)
plt.xlabel('Fecha', fontsize=12)
plt.ylabel('Cantidad vendida', fontsize=12)
plt.legend(title='Productos', loc='upper left', bbox_to_anchor=(1, 1))
plt.tight_layout()
plt.show()
```



Análisis XYZ

A partir del dataset con los datos limpios comenzamos con el análisis XYZ, el cual nos permitirá conocer la variabilidad de la demanda de los productos a través del tiempo, y clasificarlos según que tan variable es el volumen de ventas y de esta forma identificar posibles patrones o tendencias en la demanda.

En primer lugar, calcularemos la suma de las ventas (quantity) de cada producto por mes, y luego, obtendremos las ventas totales y calcularemos el promedio mensual para cada producto.

Después se agregan dos columnas nuevas 'Desviación Estandar' que nos indica la variabilidad en las ventas mensuales y 'Coeficiente de variación' (CV) que nos indica la dispersión de las ventas.

Por último, a partir del CV se obtiene la clase de cada producto:

- **Clase X:** Productos con demanda constante y predecible. ($CV \leq 50\%$).
- **Clase Y:** Productos con demanda algo variable. ($50\% < CV \leq 100\%$).
- **Clase Z:** Productos con demanda muy errática. ($CV > 100\%$).

Quedándonos el código de la siguiente manera:

```
import pandas as pd

bakery_sales = pd.read_csv('./Bakery sales- Clean.csv')

# Agregar una columna de mes
bakery_sales['date'] = pd.to_datetime(bakery_sales['date'])
bakery_sales['month'] = bakery_sales['date'].dt.month

# Agrupar por producto y mes, y calcular la suma de ventas
monthly_sales = bakery_sales.groupby(['article',
    'month'])['quantity'].sum().unstack(fill_value=0)

# Calcular el total y el promedio de ventas
monthly_sales['TOTAL'] = monthly_sales.sum(axis=1)
monthly_sales['Promedio de Ventas'] = monthly_sales['TOTAL'] / 12

# Calcular la desviación estándar por fila (producto)
monthly_sales['Desviación Estandar'] = monthly_sales.iloc[:, :12].std(axis=1,
    ddof=0)

# Calcular el coeficiente de variación correctamente
monthly_sales['Coeficiente de variación'] = (monthly_sales['Desviación
    Estandar'] / monthly_sales['Promedio de Ventas'])*100

# Clasificar productos en XYZ
def classify_xyz(cv):
    if cv <= 50:
        return 'X'
    elif cv <= 100:
        return 'Y'
    else:
        return 'Z'

monthly_sales['XYZ'] = monthly_sales['Coeficiente de
    variación'].apply(classify_xyz)

# Resetear el índice para darle formato final a la tabla
monthly_sales.reset_index(inplace=True)

monthly_sales.to_csv('ANALISIS XYZ.csv', index=False)
```


A partir del resultado de la ejecución obtenemos los siguientes resultados:

- **Clase X:** 27 productos con demanda constante y predecible.
- **Clase Y:** 42 productos con demanda algo variable.
- **Clase Z:** 78 productos con demanda muy errática.

Demanda de ingredientes

Ahora debemos obtener la demanda diaria de los ingredientes de los productos de la clase A, para ello vamos a tomar como ingredientes principales los siguientes:

- Harina (en gramos).
- Agua (en mililitros).
- Levadura (en gramos).
- Sal (en gramos).
- Azúcar (en gramos).
- Manteca (en gramos).
- Leche (en mililitros).
- Chocolate (en gramos).

Y utilizaremos las siguientes cantidades como receta para los productos que pertenecen a la clase A:

article	harina	agua	leche	levadura	sal	manteca	azucar	chocolate
TRADITIONAL BAGUETTE	300	200	0	5	5	0	0	0
FORMULE SANDWICH	250	150	0	4	3	20	5	0
CROISSANT	250	0	150	5	3	100	30	0
PAIN AU CHOCOLAT	250	0	150	5	3	100	30	250
BANETTE	300	220	0	6	4	0	0	0
BAGUETTE	300	200	0	5	5	0	0	0
SPECIAL BREAD	300	180	0	5	5	20	10	0
SANDWICH COMPLET	250	150	0	4	3	20	5	0
TRAITEUR	200	100	0	4	3	50	20	0
GRAND FAR BRETON	150	0	500	0	2	50	100	0
TARTELETTE	200	0	100	0	2	50	50	0
BRIOCHE	300	0	150	5	4	80	40	0
VIK BREAD	300	250	0	6	5	0	0	0
CEREAL BAGUETTE	300	220	0	5	5	0	0	0
GD KOUIGN AMANN	300	0	0	0	2	150	80	0
CAMPAGNE	350	250	0	6	6	0	0	0
BOULE 400G	400	280	0	8	6	0	0	0
MOISSON	300	220	0	5	5	0	0	0
ECLAIR	200	0	200	0	3	80	50	125
SAND JB EMMENTAL	250	150	0	4	3	20	5	0
COMPLET	350	250	0	6	5	0	0	0
KOUIGN AMANN	300	0	0	0	2	150	80	0

PAIN BANETTE	300	200	0	5	5	0	0	0
DIVERS VIENNOISERIE	250	0	150	5	3	100	30	0
FINANCIER X5	150	0	0	0	1	100	80	0
PAIN AUX RAISINS	250	0	150	5	3	100	30	0

A partir de esta tabla podemos escribir el siguiente código para generar el dataset de demanda de ingredientes y a partir de ahí, podemos calcular la demanda total de ingredientes diarios y luego graficarla.

```
import pandas as pd

# Cargar los datos
product_revenue = pd.read_csv('./Analisis ABC.csv')
bakery_sales = pd.read_csv('./Bakery sales- Clean.csv')

# Filtrar productos de clase A
class_a_products = product_revenue[product_revenue['ABC_class'] == 'A']['article']
class_a_sales = bakery_sales[bakery_sales['article'].isin(class_a_products)]

# Definir las recetas de los productos clase A
recipes = {
    "TRADITIONAL BAGUETTE": {"harina": 300, "agua": 200, "leche": 0,
    "levadura": 5, "sal": 5, "manteca": 0, "azucar": 0, "chocolate": 0},
    "FORMULE SANDWICH": {"harina": 250, "agua": 150, "leche": 0, "levadura":
    4, "sal": 3, "manteca": 20, "azucar": 5, "chocolate": 0},
    "CROISSANT": {"harina": 250, "agua": 0, "leche": 150, "levadura": 5,
    "sal": 3, "manteca": 100, "azucar": 30, "chocolate": 0},
    "PAIN AU CHOCOLAT": {"harina": 250, "agua": 0, "leche": 150, "levadura":
    5, "sal": 3, "manteca": 100, "azucar": 30, "chocolate": 250},
    "BANETTE": {"harina": 300, "agua": 220, "leche": 0, "levadura": 6, "sal":
    4, "manteca": 0, "azucar": 0, "chocolate": 0},
    "BAGUETTE": {"harina": 300, "agua": 200, "leche": 0, "levadura": 5, "sal":
    5, "manteca": 0, "azucar": 0, "chocolate": 0},
    "SPECIAL BREAD": {"harina": 300, "agua": 180, "leche": 0, "levadura": 5,
    "sal": 5, "manteca": 20, "azucar": 10, "chocolate": 0},
    "SANDWICH COMPLET": {"harina": 250, "agua": 150, "leche": 0, "levadura":
    4, "sal": 3, "manteca": 20, "azucar": 5, "chocolate": 0},
    "TRAITEUR": {"harina": 200, "agua": 100, "leche": 0, "levadura": 4, "sal":
    3, "manteca": 50, "azucar": 20, "chocolate": 0},
    "GRAND FAR BRETON": {"harina": 150, "agua": 0, "leche": 500, "levadura":
    0, "sal": 2, "manteca": 50, "azucar": 100, "chocolate": 0},
    "TARTELETTE": {"harina": 200, "agua": 0, "leche": 100, "levadura": 0,
    "sal": 2, "manteca": 50, "azucar": 50, "chocolate": 0},
    "BRIOCHE": {"harina": 300, "agua": 0, "leche": 150, "levadura": 5, "sal":
    4, "manteca": 80, "azucar": 40, "chocolate": 0},
    "VIK BREAD": {"harina": 300, "agua": 250, "leche": 0, "levadura": 6,
    "sal": 5, "manteca": 0, "azucar": 0, "chocolate": 0},
```

```

    "CEREAL BAGUETTE": {"harina": 300, "agua": 220, "leche": 0, "levadura": 5,
    "sal": 5, "manteca": 0, "azucar": 0, "chocolate": 0},
    "GD KOUIGN AMANN": {"harina": 300, "agua": 0, "leche": 0, "levadura": 0,
    "sal": 2, "manteca": 150, "azucar": 80, "chocolate": 0},
    "CAMPAGNE": {"harina": 350, "agua": 250, "leche": 0, "levadura": 6, "sal":
    6, "manteca": 0, "azucar": 0, "chocolate": 0},
    "BOULE 400G": {"harina": 400, "agua": 280, "leche": 0, "levadura": 8,
    "sal": 6, "manteca": 0, "azucar": 0, "chocolate": 0},
    "MOISSON": {"harina": 300, "agua": 220, "leche": 0, "levadura": 5, "sal":
    5, "manteca": 0, "azucar": 0, "chocolate": 0},
    "ECLAIR": {"harina": 200, "agua": 0, "leche": 200, "levadura": 0, "sal":
    3, "manteca": 80, "azucar": 50, "chocolate": 125},
    "SAND JB EMMENTAL": {"harina": 250, "agua": 150, "leche": 0, "levadura":
    4, "sal": 3, "manteca": 20, "azucar": 5, "chocolate": 0},
    "COMPLET": {"harina": 350, "agua": 250, "leche": 0, "levadura": 6, "sal":
    5, "manteca": 0, "azucar": 0, "chocolate": 0},
    "KOUIGN AMANN": {"harina": 300, "agua": 0, "leche": 0, "levadura": 0,
    "sal": 2, "manteca": 150, "azucar": 80, "chocolate": 0},
    "PAIN BANETTE": {"harina": 300, "agua": 200, "leche": 0, "levadura": 5,
    "sal": 5, "manteca": 0, "azucar": 0, "chocolate": 0},
    "DIVERS VIENNOISERIE": {"harina": 250, "agua": 0, "leche": 150,
    "levadura": 5, "sal": 3, "manteca": 100, "azucar": 30, "chocolate": 0},
    "FINANCIER X5": {"harina": 150, "agua": 0, "leche": 0, "levadura": 0,
    "sal": 1, "manteca": 100, "azucar": 80, "chocolate": 0},
    "PAIN AUX RAISINS": {"harina": 250, "agua": 0, "leche": 150, "levadura":
    5, "sal": 3, "manteca": 100, "azucar": 30, "chocolate": 0},
    }

# Calcular la demanda de ingredientes para todos los productos clase A
ingredients = ['harina', 'agua', 'leche', 'levadura', 'sal', 'manteca',
'azucar', 'chocolate']
class_a_sales['date'] = pd.to_datetime(class_a_sales['date'])
for ingredient in ingredients:
    class_a_sales[ingredient] = class_a_sales.apply(
        lambda row: recipes.get(row['article'], {}).get(ingredient, 0) *
row['quantity'], axis=1
    )

# Agrupar la demanda diaria
daily_ingredient_demand =
class_a_sales.groupby('date')[ingredients].sum().reset_index()
daily_ingredient_demand.to_csv('demanda_diaria.csv', index=False)

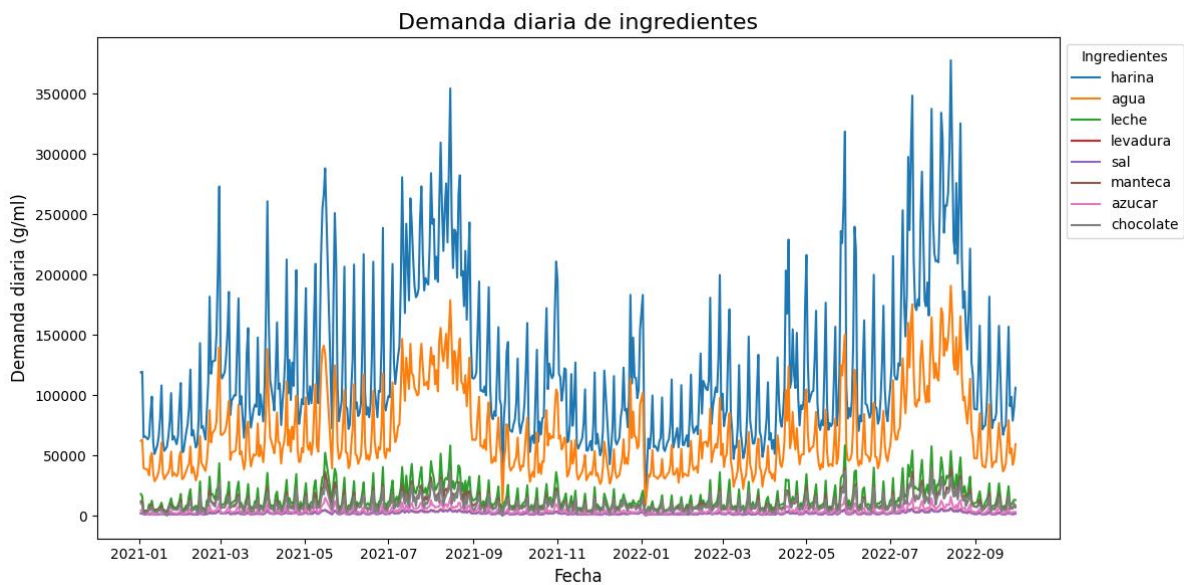
# Agrupar la demanda semanal
class_a_sales['week'] = class_a_sales['date'].dt.to_period('W')
weekly_ingredient_demand =
class_a_sales.groupby('week')[ingredients].sum().reset_index()
weekly_ingredient_demand.to_csv('demanda_semanal.csv', index=False)

```

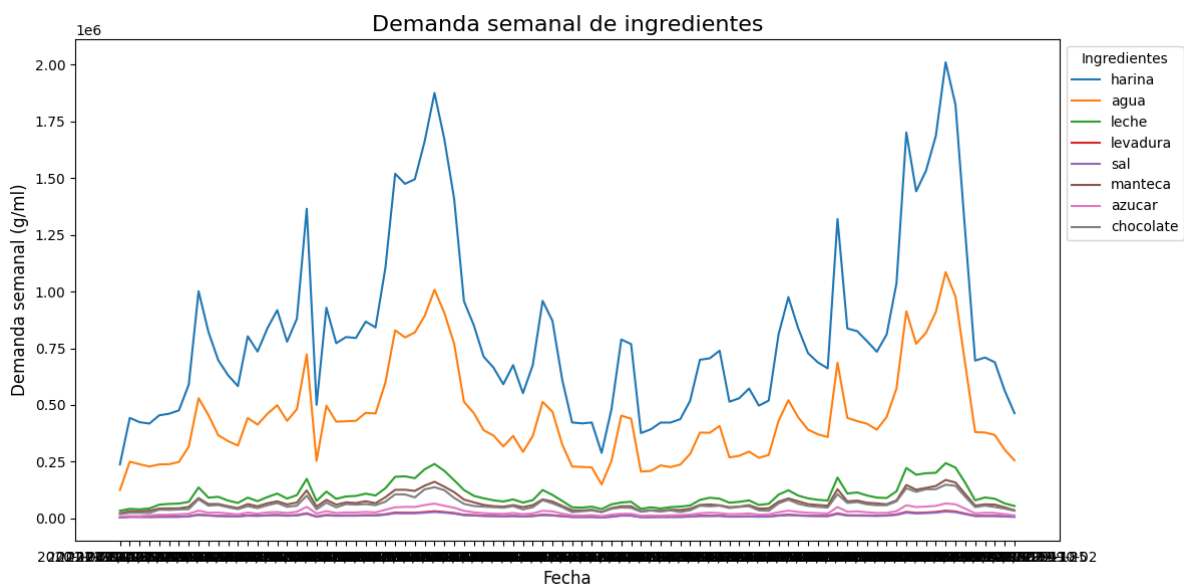
```
# Agrupar la demanda mensual
class_a_sales['month'] = class_a_sales['date'].dt.to_period('M')
monthly_ingredient_demand =
class_a_sales.groupby('month')[ingredients].sum().reset_index()
monthly_ingredient_demand.to_csv('demanda_mensual.csv', index=False)
```

A partir de los datasets generados podemos generar gráficos de la demanda permitiéndonos identificar tendencias.

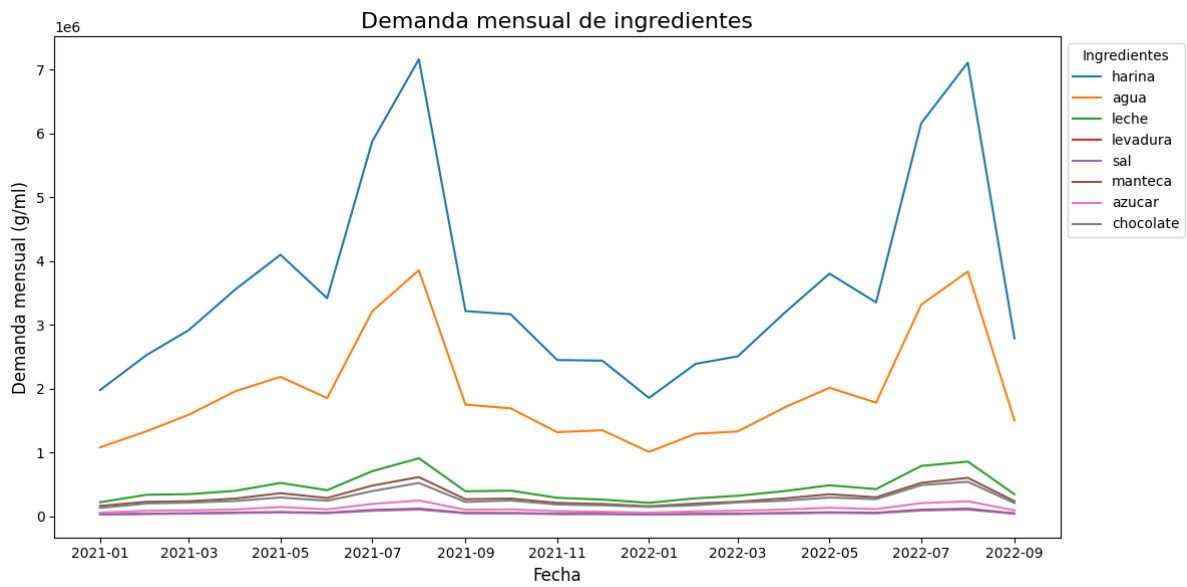
Demanda diaria de ingredientes



Demanda semanal de ingredientes



Demanda mensual de ingredientes



Modelos de pronósticos

Una vez obtenida la demanda de cada ingrediente, construimos modelos de pronóstico individuales para identificar patrones de estacionalidad en las demandas. Estos patrones serán clave para diseñar políticas de reabastecimiento más eficientes.

Para ello, aplicamos inicialmente el modelo **Holt-Winters (Exponential Smoothing)**, pero tras evaluar su desempeño, observamos que puede no capturar bien ciertas tendencias cuando la serie es demasiado compleja y que el período estacional óptimo no correspondía con la estacionalidad real de los datos. Por ello, optamos por utilizar un modelo **SARIMA (Seasonal ARIMA)**, el cual es más adecuado y generalmente proporciona mejores pronósticos en datos con estacionalidad y tendencias bien definidas.

Preprocesamiento de Datos

Debido a que la demanda semanal se muestra bastante mas suave que la demanda diaria, se consolidó el conjunto de datos en series temporales semanales para reducir la variabilidad diaria y mejorar la estabilidad del modelo. Posteriormente, se corrigió el índice para que inicie en la fecha 01-01-2021 y utilice una frecuencia semanal para facilitar el ajuste del modelo SARIMA, y se dividió el conjunto de datos en un 80% dedicado al entrenamiento del modelo, y el 20% restante dedicado a la prueba del modelo.

Configuración del Modelo

Tras diversas pruebas, el modelo final se ajustó con los siguientes parámetros

- **Orden ARIMA:** (1, 1, 2)
- **Orden estacional:** (1, 1, 0, 52)

Considerando las 52 semanas del año

Aunque para ingredientes con mayor variabilidad como la leche o el chocolate se utilizaron los siguientes parámetros:

- **Orden ARIMA:** (1, 1, 2)
- **Orden estacional:** (1, 1, 0, 52)

El modelo se utilizó para pronosticar la demanda de los próximos 52 períodos semanales (el año siguiente).

Evaluación del Modelo

Para evaluar la calidad del modelo, calculamos las siguientes métricas:

- **MAE (Mean Absolute Error)**
- **RMSE (Root Mean Squared Error)**
- **MAPE (Mean Absolute Percentage Error)**
- **Sesgo**

Adicionalmente, generamos gráficos de las series de tiempo y exportamos el pronóstico a un archivo .csv para su posterior análisis.

Código para el ingrediente 'harina'

```
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Cargar los datos
df = pd.read_csv("demanda_semanal.csv", parse_dates=["week"],
index_col="week")

# Cambiar índice a numérico
df.index = pd.date_range(start="2021-01-01", periods=len(df), freq="W")

# Seleccionar la serie temporal del ingrediente
ingrediente = "harina"
demand = df[ingrediente]

# Dividir en entrenamiento y prueba (80% entrenamiento, 20% prueba)
train_size = int(len(demand) * 0.8)
train, test = demand[:train_size], demand[train_size:]

# Ajustar modelo SARIMA
modelo = sm.tsa.SARIMAX(
    train,
    order=(1, 1, 2), # Parámetros ARIMA (ajustables)
    seasonal_order=(1, 1, 0, 52), # Estacionalidad semanal
    enforce_stationarity=False,
    enforce_invertibility=False,
)

sarima_fit = modelo.fit()

# Pronosticar en conjunto de prueba
test_forecast = sarima_fit.predict(start=len(train), end=len(train) +
len(test) - 1)

# Calcular métricas de error
mae = mean_absolute_error(test, test_forecast)
mse = mean_squared_error(test, test_forecast)
rmse = np.sqrt(mse)
mape = np.mean(np.abs((test - test_forecast) / test)) * 100
bias = np.mean(test_forecast - demand)

# Mostrar métricas
print("Métricas en conjunto de prueba:")
print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"MAPE: {mape:.2f}%")
print(f"Sesgo: {bias:.2f}")
```

```
# Pronosticar demanda futura (52 semanas)
future_forecast = sarima_fit.predict(start=len(demand), end=len(demand) + 51)

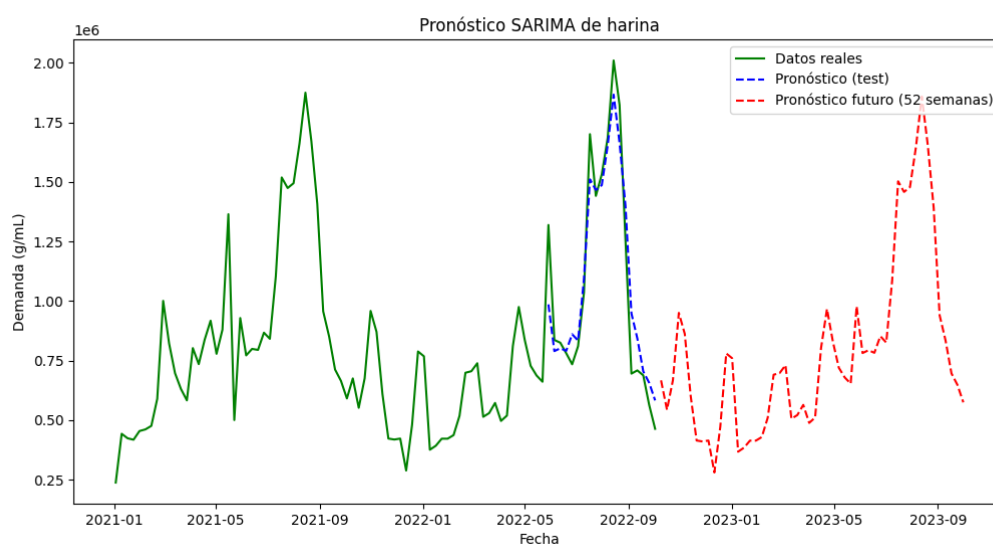
# Graficar resultados
plt.figure(figsize=(12, 6))
plt.plot(demand, label="Datos reales", color="blue")
plt.plot(test.index, test_forecast, label="Pronóstico (test)", color="orange",
linestyle="--")
plt.plot(future_forecast, label="Pronóstico futuro (52 semanas)", color="red",
linestyle="--")
plt.title(f"Pronóstico SARIMA de {ingrediente}")
plt.xlabel("Semanas")
plt.ylabel("Demanda semanal (kg/L)")
plt.legend()
plt.show()

# Exportar pronóstico a CSV
df_forecast = pd.DataFrame({"date": future_forecast.index, "forecast":
future_forecast.values})
df_forecast.to_csv(f"PRONOSTICO_{ingrediente}_SARIMA.csv", index=False)
```

Pronósticos

A continuación, se indicarán los pronósticos obtenidos para la demanda de los distintos ingredientes:

Pronóstico de demanda de Harina



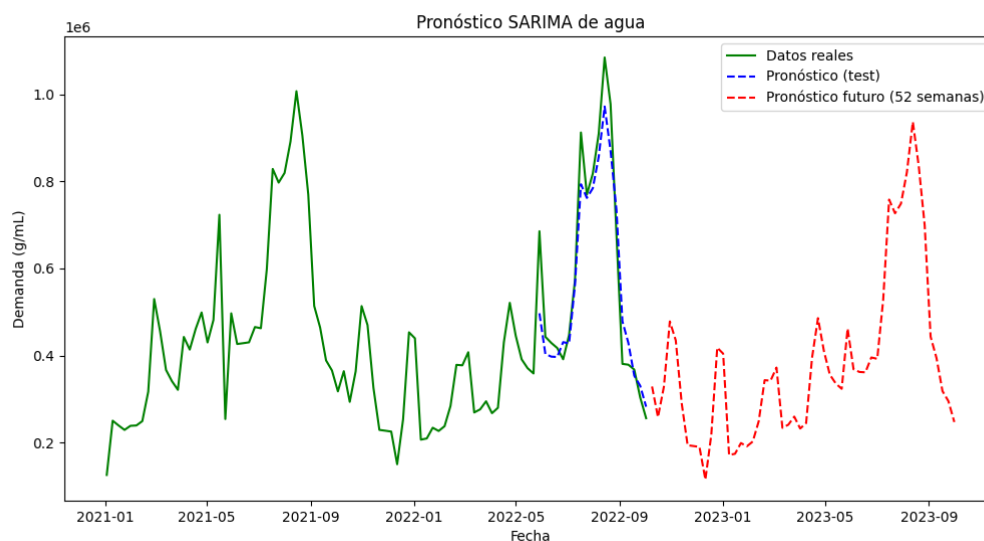
Métricas en conjunto de prueba:

- MAE: 104063.51

- RMSE: 134865.44
- MAPE: 10.83%
- Sesgo: 1550.61

El modelo muestra un buen desempeño en la predicción de la demanda de harina semanal. Aunque los errores existen, los valores son manejables en comparación con el consumo total semanal (una bolsa de harina contiene 25 kg). El pronóstico es confiable para la planificación de insumos semanales.

Pronóstico de demanda de Agua

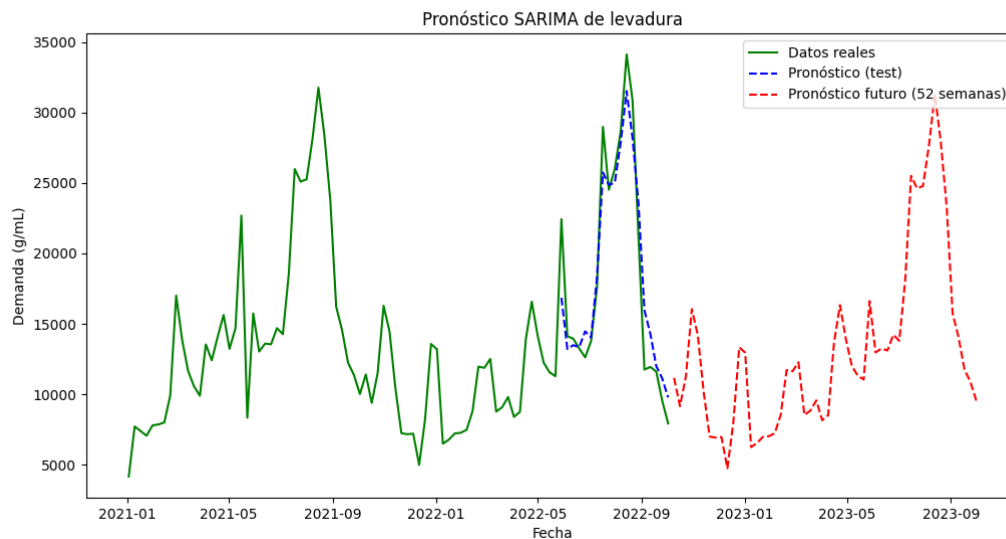


Métricas en conjunto de prueba:

- MAE: 55563.97
- RMSE: 72563.68
- MAPE: 9.51%
- Sesgo: -24202.72

El pronóstico de la demanda semanal de agua es preciso, y los errores, en términos de mililitros, no representan una variabilidad crítica. Este modelo es adecuado para la gestión del abastecimiento semanal.

Pronóstico de demanda de Levadura

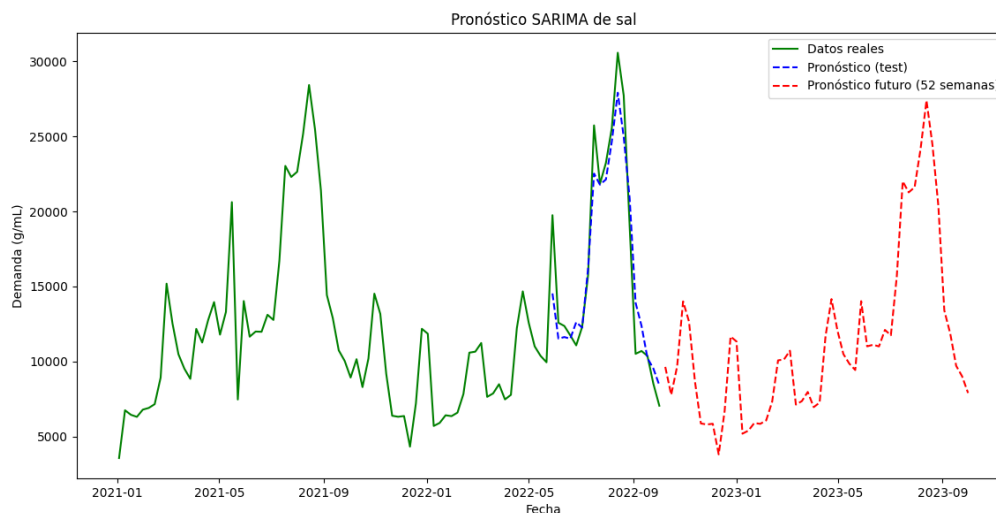


Métricas en conjunto de prueba:

- MAE: 1749.72
- RMSE: 2262.67
- MAPE: 10.59%
- Sesgo: -64.08

El modelo muestra un buen desempeño en la predicción de la demanda semanal de levadura. Los errores son manejables y el sesgo es prácticamente nulo, lo que indica que el pronóstico es confiable para la planificación de insumos semanales.

Pronóstico de demanda de Sal



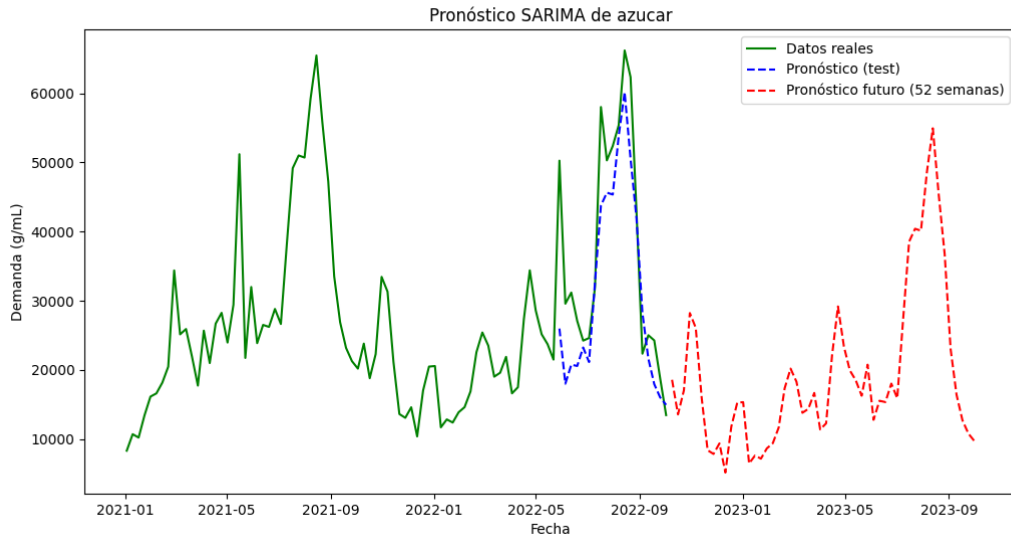
Métricas en conjunto de prueba:

- MAE: 1546.17
- RMSE: 2041.31
- MAPE: 9.98%

- Sesgo: -359.87

El modelo presenta un buen desempeño en la predicción de la demanda semanal de sal. Los errores se mantienen en niveles aceptables y el sesgo es bajo, lo que permite confiar en el pronóstico para la planificación de insumos semanales.

Pronóstico de demanda de Azúcar

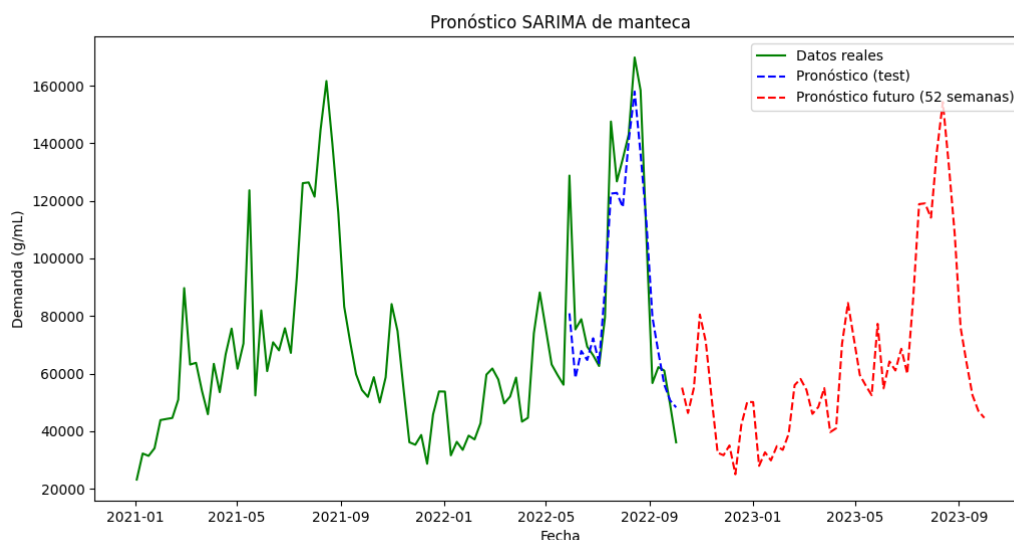


Métricas en conjunto de prueba:

- MAE: 6534.24
- RMSE: 8696.94
- MAPE: 17.75%
- Sesgo: -5681.59

El modelo muestra una precisión moderada en la predicción de la demanda semanal de azúcar, en comparación con otros ingredientes, lo que indica una mayor variabilidad. Aun así, los errores se mantienen dentro de un rango aceptable, superando apenas 1 kg diario, por lo que el pronóstico sigue siendo útil para la planificación de insumos semanales.

Pronóstico de demanda de Manteca

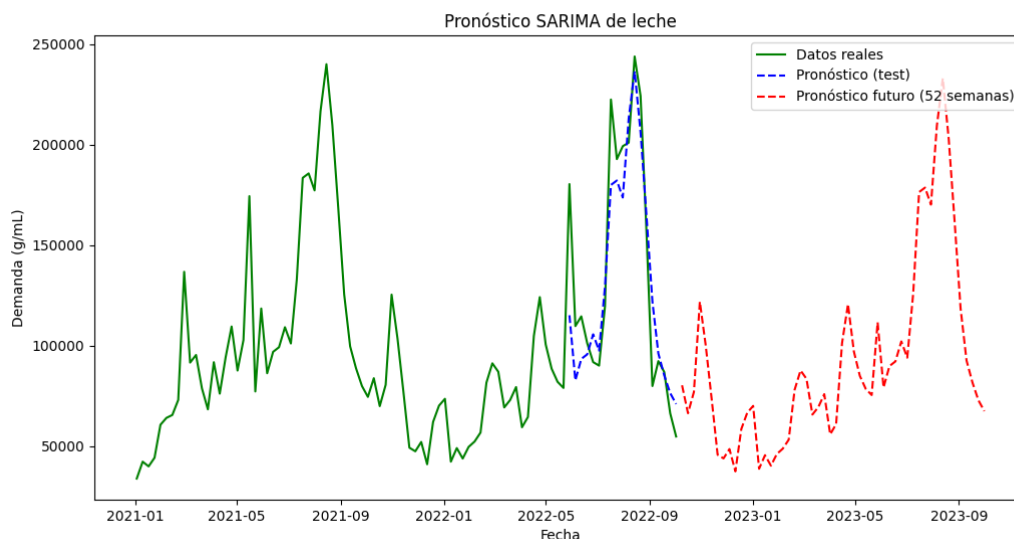


Métricas en conjunto de prueba:

- MAE: 12179.25
- RMSE: 16493.91
- MAPE: 13.53%
- Sesgo: -5426.86

El modelo muestra una precisión adecuada en la predicción de la demanda semanal de manteca. Aunque los errores son algo más elevados que otros ingredientes, se mantienen dentro de un rango manejable, ya que un paquete de manteca para panadería puede ser de hasta 25 kg. El pronóstico sigue siendo útil para la planificación de insumos semanales.

Pronóstico de demanda de Leche



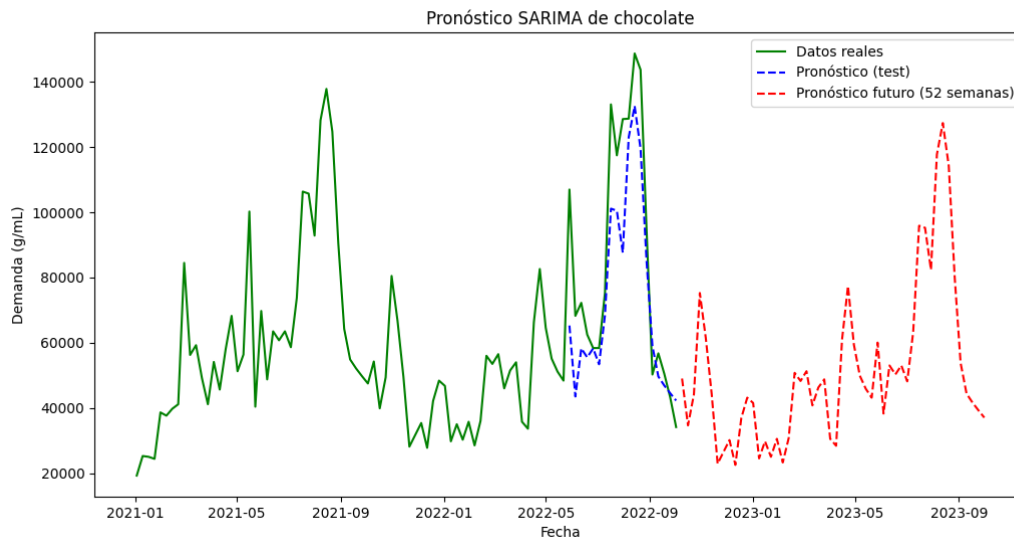
Métricas en conjunto de prueba:

- MAE: 18319.33

- RMSE: 24179.38
- MAPE: 14.68%
- Sesgo: -5332.81

El modelo presenta dificultades en la predicción de la demanda de leche, los errores son relativamente altos, lo que indica que el pronóstico de leche podría ser menos confiable para la planificación de insumos semanales en comparación con otros ingredientes. Sin embargo, el pronóstico puede servir como referencia.

Pronóstico de demanda de Chocolate



Métricas en conjunto de prueba:

- MAE: 14437.52
- RMSE: 18985.29
- MAPE: 15.81%
- Sesgo: -12525.53

Al igual que la leche, el modelo presenta dificultades en la predicción de la demanda de chocolate. Aunque el modelo presenta algunas dificultades, los errores son aceptables en términos relativos, por lo que el pronóstico aún puede servir como una referencia útil para la planificación de insumos semanales.

En conclusión, el modelo permitió identificar patrones de tendencia y estacionalidad en la demanda de ingredientes clave, lo que facilita la planificación del reabastecimiento. Los resultados muestran que el modelo tiene un desempeño aceptable en la mayoría de los ingredientes.

Patrones de estacionalidad

De acuerdo con el punto anterior, podemos analizar las fluctuaciones en las cantidades demandadas a lo largo del tiempo, y de esta forma identificar patrones de estacionalidad en las demandas de los ingredientes. Los patrones identificados son los siguientes:

Los ingredientes de panificación básicas, como harina, agua, levadura y sal muestran una demanda relativamente estable a lo largo del año, con picos notables en fechas específicas como Navidad (diciembre) y Semana Santa (abril). Esto sugiere un aumento en la producción de productos horneados durante estas festividades.

La demanda de ingredientes como leche, manteca y azúcar presenta fluctuaciones más pronunciadas, con aumentos significativos durante los meses de verano (junio a agosto) y en diciembre. Esto podría estar relacionado con la producción de productos de pastelería y postres.

La demanda de chocolate muestra picos claros en diciembre, coincidiendo con las festividades navideñas, y en febrero, posiblemente relacionado con el Día de San Valentín. Estos patrones de estacionalidad pueden ser útiles para la planificación de inventarios y la optimización de la producción, asegurando que se disponga de suficientes ingredientes durante los períodos de mayor demanda. Además, pueden ayudar a anticipar aumentos en la producción de ciertos productos durante las temporadas altas.

Modelos de inventarios

Para comenzar con el modelo de inventarios, en primer lugar, debemos definir los parámetros que se utilizarán, siendo estos:

Costo de adquisición

Luego de una investigación en una panadería de Concepción del Uruguay, los costos de adquisición son aproximadamente los siguientes:

- **Harina:** \$460 / kg
- **Agua:** \$0.30 / L
- **Leche:** \$1300 / L
- **Levadura:** \$7400 / kg
- **Sal:** \$475 / kg
- **Manteca:** \$3200 / kg
- **Azúcar:** \$650 / kg
- **Chocolate:** \$13000 / kg

Volumen ocupado y capacidad total de almacenamiento:

Para obtener el valor de algunos parámetros y tener en cuenta la restricción de la capacidad total de almacenamiento debemos conocer que volumen ocupa cada ingrediente dentro del depósito. Para esto vamos a tener en cuenta algunas consideraciones:

- Los ingredientes secos (harina, sal, azúcar) se almacenan en un depósito de 2m x 2m x 2m, siendo el volumen máximo 8 m³.
- Los ingredientes húmedos (leche, manteca, levadura, chocolate) se almacenan en una heladera, con un volumen máximo de 1 m³.
- El ingrediente Agua proviene de la red por lo que no ocupa espacio en almacenamiento.

Quedando así el volumen ocupado por cada ingrediente de la siguiente manera:

Ingrediente	Tipo	Presentación	Volumen por unidad (m ³)
Harina	Seco	Bolsa 25 kg	0.0525 m ³
Sal	Seco	Bolsa 20 kg	0.040 m ³
Azúcar	Seco	Bolsa 25 kg	0.045 m ³
Levadura	Húmedo	Pack 500 g	0.00125 m ³
Leche	Húmedo	Botellas 1L	0.00097 m ³
Manteca	Húmedo	Pack 1 kg	0.00109 m ³
Chocolate	Húmedo	Pack 1 kg	0.00077 m ³
Agua	-	-	-

Costo de ordenar

El costo de ordenar representa el gasto fijo que se realiza cada vez que se realiza un pedido de ingredientes, en este caso, los proveedores cobran el envío de la orden y además, una vez que se recibe un pedido un empleado se encarga de recibir y verificar los insumos, por lo que el costo de ordenar en promedio es de **\$150000**.

Tasa de inmovilización de capital y límite de capital inmovilizado

La tasa de inmovilización de capital representa la proporción del capital de la panadería que está invertido en el inventario, para este caso vamos a determinar una tasa del **10% mensual**.

Por otro lado, el límite de capital inmovilizado define la cantidad máxima de dinero que puede mantenerse en inventario sin afectar la liquidez del negocio. Para este caso, se establece un límite de **\$500000**.

Costos de almacenamiento

El costo de almacenamiento incluye el costo de mantener el stock en el depósito o la heladera, en este caso vamos a considerar el factor inmovilización del capital, por lo que para calcular el costo de almacenamiento se considera una tasa de inmovilización semanal del 2,3% sobre el costo de adquisición, correspondiente a una tasa mensual del 10%, quedando:

- **Harina:** \$10.62 / kg
- **Agua:** -
- **Leche:** \$30.02 / L
- **Levadura:** \$170.90 / kg
- **Sal:** \$10.97 / kg
- **Manteca:** \$73.92 / kg
- **Azúcar:** \$15.01 / kg
- **Chocolate:** \$300 / kg

Demanda

Para determinar la cantidad de ingredientes a reabastecer, se debe conocer la demanda de cada uno. En el punto anterior, obtuvimos la demanda de cada producto, para el modelo de inventarios, la demanda utilizada en el modelo corresponde a valores semanales para cada ingrediente, expresados en kilogramos, obtenidos a partir de los pronósticos procesados desde el dataset original.

Penalización por ordenar

Dado que el modelo fue formulado como un problema **no lineal continuo**, se decidió **no utilizar variables binarias** para modelar los pedidos. En su lugar, se empleó una función de penalización suave, que permite capturar el efecto económico de realizar un pedido sin

necesidad de programación entera o mixta. La función para la penalización por pedidos queda de la siguiente manera:

$$\text{Penalización} = 1 - e^{-\alpha_i \cdot Q_{it}}$$

Donde:

- K es el costo fijo por pedido (en \$).
- Q_{it} es la cantidad pedida del ingrediente i en la semana t .
- α_i es un parámetro calibrado para cada ingrediente.

El valor de α_i se calcula a partir de un **lote de referencia** Q_{ref} como:

$$\alpha_i = \frac{-\ln(0.05)}{Q_{ref_i}}$$

El Q_{ref_i} es el promedio de la demanda obtenidos en el modelo inicial.

Esta función tiene un comportamiento creciente y asintótico:

- Si $Q_{it} \rightarrow 0$, el costo tiende a 0
- Si $Q_{it} \rightarrow \infty$, el costo tiende a K
- Para un valor intermedio $Q_{it} = Q_{ref_i}$ el costo alcanza el 95% de K .

Resumen de parámetros

Parámetro	Harina	Agua	Leche	Levadura	Sal	Manteca	Azúcar	Chocolate
Costo de adquisición	\$460 / kg	\$0.30 / L	\$1300 / L	\$7400 / kg	\$475 / kg	\$3200 / kg	\$650 / kg	\$13,000 / kg
Costo de almacenamiento	\$10.62 / kg	-	\$30.02 / L	\$170.90 / kg	\$10.97 / kg	\$73.92 / kg	\$15.01 / kg	\$300 / kg
Demanda	Tabla de demandas (Archivo .csv)							
Volumen ocupado	0.0525/25 m ³ /kg	-	0.040 m ³ /L	0.045/0.5 m ³ /kg	0.04/20 m ³ /kg	0.00097 m ³ /kg	0.045/25 m ³ /kg	0.00077 m ³
Costo de ordenar	\$150000							
Tasa de inmovilización de capital	10%							
Capacidad total de almacenamiento	8 m ³	-	1 m ³	1 m ³	8 m ³	1 m ³	8 m ³	1 m ³
Límite de capital inmovilizado	\$500000							
Q_{ref}	707.38 kg	-	73.17 l	11.93 kg	10.27 kg	59.25 kg	19.38 kg	51.22 kg
α	0.004235	-	0.040939	0.251214	0.291555	0.050561	0.154455	0.058479

Modelo de programación no lineal

Para la gestión del inventario de la panadería, vamos a implementar un modelo de programación no lineal, el objetivo es minimizar los costos totales de inventario,

considerando los costos de adquisición, almacenamiento, y de orden, considerando las restricciones de capacidad de almacenamiento y límite de capital inmovilizado.

Variables de decisión

- Q_{it} : Cantidad a ordenar del ingrediente i en el periodo t (en kg o litros, dependiendo del ingrediente).
- I_{it} : Nivel de inventario del ingrediente i al finalizar el periodo t (en kg o litros, dependiendo del ingrediente).

Parámetros

- D_{it} : Demanda mensual del ingrediente i en el periodo t (en kg o litros).
- K : Costo de orden por pedidos (en \$).
- C_i : Costo de adquisición del ingrediente i (en \$/kg o \$/litro).
- h_i : Costo de almacenamiento del ingrediente i (en \$/kg o \$/litro).
- L : Tiempo de entrega (en días).
- V_i : Volumen ocupado por unidad del ingrediente i (en m³).
- $V_{max-secos}$: Capacidad total de almacenamiento (en m³).
- $V_{max-humedos}$: Capacidad total de almacenamiento (en m³).
- C_{max} : Límite de capital inmovilizado (en \$).
- α_i : Parámetro de penalización para cada ingrediente, calibrado de forma que se pague el 95% de K al ordenar una cantidad de referencia $Q_{ref,i}$

Función objetivo

Buscamos minimizar los costos, por lo que queda de la siguiente manera:

$$MIN Z = \sum_t \sum_i \underbrace{C_i \cdot Q_{it}}_{\text{Costos de adquisición}} + \underbrace{h_{it} \cdot I_{it}}_{\text{Costos de Almacenamiento}} + \underbrace{K \cdot (1 - e^{-\alpha_i Q_{it}})}_{\text{Penalización por ordenar}}$$

Restricciones

- **Balance de inventario:** nos debemos asegurar de que se mantenga el inventario sobrante de un periodo a otro.

$$I_{it} = I_{it-1} + Q_{it} - D_{it} \quad \forall i, t$$

- **Restricciones de almacenamiento:** nos debemos asegurar que el volumen total de los ingredientes no exceda la capacidad de almacenamiento.

$$\sum_i V_i \cdot I_{it} \leq V_{max-secos} \quad \forall i \in Secos$$

$$\sum_i V_i \cdot I_{it} \leq V_{max-humedos} \quad \forall i \in Humedos$$

- **Restricciones de capital inmovilizado:** nos debemos asegurar que el capital invertido en inventario no exceda el límite establecido.

$$\sum_i C_i \cdot I_{it} \leq C_{max} \quad \forall t$$

- **Restricciones de demanda:** nos debemos asegurar que la cantidad a ordenar sea superior a la demanda.

$$Q_{i_t} + I_{i_t} \geq D_{i_t}$$

- **Restricción de no negatividad:**

$$Q_{i_t} \geq 0, I_{i_t} \geq 0$$

Resultados

A continuación se presenta el código Python utilizado para resolver el modelo de programación no lineal mediante la biblioteca **Pyomo** y el solver **IPOPT**.

```
import pandas as pd
from math import log
from pyomo.environ import *

# ----- DATOS DE ENTRADA -----

# Cargar el archivo de pronóstico (en gramos) y convertir a kg
data = pd.read_csv('PRONOSTICO_UNIFICADO.csv')
data = data.drop(columns=['agua'], errors='ignore')
data = data.head(4) # Horizonte de 4 semanas

demanda = {(ingrediente, t + 1): data.iloc[t][ingrediente] / 1000
            for t in range(len(data)) for ingrediente in data.columns[1:]}

ingredientes = list(data.columns[1:])
periodos = range(1, 5)

# ----- PARÁMETROS -----

# Costos
C = {'harina': 460, 'leche': 1300, 'levadura': 7400,
     'sal': 475, 'manteca': 3200, 'azucar': 650, 'chocolate': 13000}

# Costos de almacenamiento (10% mensual → semanal)
h = {i: C[i] * 0.023 for i in C}

# Volumen ocupado por kg (en m3/kg)
V = {'harina': 0.0525/25, 'sal': 0.04/20, 'azucar': 0.045/25,
     'leche': 0.04, 'levadura': 0.045/0.5, 'manteca': 0.00097, 'chocolate':
     0.00077}

# Capacidad de almacenamiento (m3) y capital inmovilizado ($)
Vmax_secos = 8
Vmax_humedos = 1
Cmax = 5e8

# Costos fijos aproximados
```

```

K = 150000
Qref = {'harina': 707.38, 'leche': 73.17, 'levadura': 11.93,
        'sal': 10.27, 'manteca': 59.25, 'azucar': 19.38, 'chocolate': 51.22}
alpha = {i: -log(0.05) / Qref[i] for i in ingredientes}

# ----- MODELO PYOMO -----

model = ConcreteModel()
model.ingredientes = Set(initialize=ingredientes)
model.periodos = Set(initialize=periodos)

# Variables
model.Q = Var(model.ingredientes, model.periodos, domain=NonNegativeReals)
model.S = Var(model.ingredientes, model.periodos, domain=NonNegativeReals)

# Balance de inventario
def balance_inventario(model, i, t):
    if t == 1:
        return model.S[i, t] == model.Q[i, t] - demanda[i, t]
    else:
        return model.S[i, t] == model.S[i, t - 1] + model.Q[i, t] - demanda[i,
t]
model.balance_inventario = Constraint(model.ingredientes, model.periodos,
rule=balance_inventario)

# Capacidad de almacenamiento
secos = ['harina', 'sal', 'azucar']
humedos = ['leche', 'levadura', 'manteca', 'chocolate']

def capacidad_secos_rule(model, t):
    return sum(V[i] * model.S[i, t] for i in secos) <= Vmax_secos

def capacidad_humedos_rule(model, t):
    return sum(V[i] * model.S[i, t] for i in humedos) <= Vmax_humedos

model.capacidad_secos = Constraint(model.periodos, rule=capacidad_secos_rule)
model.capacidad_humedos = Constraint(model.periodos,
rule=capacidad_humedos_rule)

# Capital inmovilizado
def capital_inmovilizado_rule(model, t):
    return sum(C[i] * model.S[i, t] for i in model.ingredientes) <= Cmax
model.capital_inmovilizado = Constraint(model.periodos,
rule=capital_inmovilizado_rule)

# Función objetivo: minimizar costos totales

def objetivo_rule(model):

```

```

costos_orden = sum(K * (1 - exp(-alpha[i] * model.Q[i, t]))
                    for i in model.ingredientes for t in model.periodos)
costos_compra = sum(C[i] * model.Q[i, t] for i in model.ingredientes for t
in model.periodos)
costos_stock = sum(h[i] * model.S[i, t] for i in model.ingredientes for t
in model.periodos)
return costos_orden + costos_compra + costos_stock

model.objetivo = Objective(rule=objetivo_rule, sense=minimize)

# ----- SOLVER -----

solver = SolverFactory('ipopt')
solver.options['tol'] = 1e-6
solver.options['max_iter'] = 10000
resultado = solver.solve(model, tee=True)

# ----- RESULTADOS -----

print("\n==== PLAN DE PEDIDOS E INVENTARIOS (kg) =====")
for i in model.ingredientes:
    for t in model.periodos:
        q = model.Q[i, t].value
        s = model.S[i, t].value
        print(f"{i:10s} semana {t}: Q = {q:8.1f} S = {s:8.1f}")

```

Y los resultados obtenidos fueron los siguientes:

Harina		
	Cantidad a ordenar	Inventario
Semana 1	1878.7	1211.5
Semana 2	0	667.7
Semana 3	0	0
Semana 4	950.7	0

Leche		
	Cantidad a ordenar	Inventario
Semana 1	88.9	21.9
Semana 2	31.3	0
Semana 3	63.8	0
Semana 4	108.6	0

Levadura		
	Cantidad a ordenar	Inventario
Semana 1	11.2	0
Semana 2	9.2	0

Semana 3	11.3	0
Semana 4	16.0	0

Sal		
	Cantidad a ordenar	Inventario
Semana 1	41.1	31.5
Semana 2	0	23.7
Semana 3	0	14.0
Semana 4	0	0

Manteca		
	Cantidad a ordenar	Inventario
Semana 1	156.5	101.4
Semana 2	0	55.1
Semana 3	0	0
Semana 4	80.5	0

Azúcar		
	Cantidad a ordenar	Inventario
Semana 1	77.5	58.9
Semana 2	0	45.3
Semana 3	0	28.2
Semana 4	0	0

Chocolate		
	Cantidad a ordenar	Inventario
Semana 1	84.6	35.1
Semana 2	0	0
Semana 3	44.6	0
Semana 4	75.7	0

El modelo muestra un patrón de reabastecimiento coherente, con algunos productos reponiéndose en un solo lote (ej. sal, azúcar), mientras que otros como leche y chocolate requieren pedidos más frecuentes debido a la baja capacidad de almacenamiento de húmedos.

Análisis de sensibilidad

Con el objetivo de evaluar la robustez del modelo y comprender cómo varían las decisiones óptimas ante cambios en los parámetros clave, se realizó un análisis de sensibilidad sobre los siguientes elementos:

Parámetro K

Se evaluó una modificación en costo fijo por pedido realizado, obteniendo los siguientes resultados, con un costo de \$200.000:

Harina		
	Cantidad a ordenar	Inventario
Semana 1	1878.7	1211.5
Semana 2	0	667.7
Semana 3	0	0
Semana 4	950.7	0

Sal		
	Cantidad a ordenar	Inventario
Semana 1	41.1	31.5
Semana 2	0	23.7
Semana 3	0	14.0
Semana 4	0	0

Leche		
	Cantidad a ordenar	Inventario
Semana 1	88.1	21.0
Semana 2	32.1	0
Semana 3	63.8	0
Semana 4	108.6	0

Manteca		
	Cantidad a ordenar	Inventario
Semana 1	156.5	101.4
Semana 2	0	55.1
Semana 3	0	0
Semana 4	80.5	0

Levadura		
	Cantidad a ordenar	Inventario
Semana 1	11.2	0
Semana 2	9.2	0
Semana 3	11.3	0
Semana 4	16.0	0

Azúcar		
	Cantidad a ordenar	Inventario
Semana 1	77.5	58.9
Semana 2	0	45.3
Semana 3	0	28.2
Semana 4	0	0

Chocolate		
	Cantidad a ordenar	Inventario
Semana 1	129.2	79.7
Semana 2	0	0
Semana 3	0	0
Semana 4	75.7	0

Y los siguientes resultados con un costo de \$50.000:

Harina		
	Cantidad a ordenar	Inventario
Semana 1	667.3	0.0
Semana 2	543.8	0.0
Semana 3	667.7	0.0
Semana 4	950.7	0.0

Sal		
	Cantidad a ordenar	Inventario
Semana 1	41.1	31.5
Semana 2	0.0	23.7
Semana 3	0.0	14.0
Semana 4	0.0	0.0

Leche		
	Cantidad a ordenar	Inventario
Semana 1	67.1	0.0
Semana 2	53.2	0.0
Semana 3	63.8	0.0
Semana 4	108.6	0.0

Manteca		
	Cantidad a ordenar	Inventario
Semana 1	156.5	101.4
Semana 2	0.0	55.1
Semana 3	0.0	0.0
Semana 4	80.5	0.0

Levadura		
	Cantidad a ordenar	Inventario
Semana 1	21.2	10.0
Semana 2	0.0	0.9
Semana 3	10.4	0.0
Semana 4	16.0	0.0

Azúcar		
	Cantidad a ordenar	Inventario
Semana 1	77.5	58.9
Semana 2	0.0	45.3
Semana 3	0.0	28.2
Semana 4	0.0	0.0

Chocolate		
	Cantidad a ordenar	Inventario
Semana 1	49.5	0.0
Semana 2	35.1	0.0
Semana 3	44.6	0.0
Semana 4	75.7	0.0

Esto demuestra que el modelo tiene sentido y responde de manera previsible al costo de ordenar, ya que, si aumenta el costo, el modelo prefiere almacenar más, y si disminuye el costo prefiere ordenar más y almacenar menos.

Parámetro $V_{\text{max-humedos}}$

Si variamos la capacidad máxima de la heladera, también podemos ver una variación en el plan de abastecimiento, si lo disminuimos a 0.5 m³:

Harina		
	Cantidad a ordenar	Inventario
Semana 1	1878.7	1211.5
Semana 2	0	667.7
Semana 3	0	0
Semana 4	950.7	0

Sal		
	Cantidad a ordenar	Inventario
Semana 1	41.1	31.5
Semana 2	0	23.7
Semana 3	0	14.0
Semana 4	0	0

Leche		
	Cantidad a ordenar	Inventario
Semana 1	76.4	9.4
Semana 2	43.8	0.0
Semana 3	63.8	0.0
Semana 4	108.6	0.0

Manteca		
	Cantidad a ordenar	Inventario
Semana 1	156.5	101.4
Semana 2	0	55.1
Semana 3	0	0
Semana 4	80.5	0

Levadura		
	Cantidad a ordenar	Inventario
Semana 1	11.2	0
Semana 2	9.2	0
Semana 3	11.3	0
Semana 4	16.0	0

Azúcar		
	Cantidad a ordenar	Inventario
Semana 1	77.5	58.9
Semana 2	0	45.3
Semana 3	0	28.2
Semana 4	0	0

Chocolate		
	Cantidad a ordenar	Inventario
Semana 1	49.5	0.0
Semana 2	35.1	0.0
Semana 3	44.6	0.0
Semana 4	75.7	0.0

Y con un aumento a 5 m³ obtuvimos los siguientes resultados:

Harina		
	Cantidad a ordenar	Inventario
Semana 1	1878.7	1211.5
Semana 2	0	667.7
Semana 3	0	0
Semana 4	950.7	0

Sal		
	Cantidad a ordenar	Inventario
Semana 1	41.1	31.5
Semana 2	0	23.7
Semana 3	0	14.0
Semana 4	0	0

Leche		
	Cantidad a ordenar	Inventario
Semana 1	168.3	101.3
Semana 2	0.0	48.1
Semana 3	15.6	0.0
Semana 4	108.6	0.0

Manteca		
	Cantidad a ordenar	Inventario
Semana 1	156.5	101.4
Semana 2	0	55.1
Semana 3	0	0
Semana 4	80.5	0

Levadura		
	Cantidad a ordenar	Inventario
Semana 1	20.3	9.2
Semana 2	0.0	0.0
Semana 3	11.3	0.0
Semana 4	16.0	0.0

Azúcar		
	Cantidad a ordenar	Inventario
Semana 1	77.5	58.9
Semana 2	0	45.3
Semana 3	0	28.2
Semana 4	0	0

Chocolate		
	Cantidad a ordenar	Inventario
Semana 1	84.6	35.1
Semana 2	0	0
Semana 3	44.6	0
Semana 4	75.7	0

Por lo que podemos concluir que, como se observó, aumentando la capacidad de la heladera, la leche y levadura comenzaron a acumular inventario, en cambio, disminuyendo la capacidad, el modelo forzó pedidos semanales.